

Where to put the JavaScript or external js file in your html page.

Option 1: Put your JavaScript right before the closing `</body>` tag.

Build the html and make it work. “HTML loads first, then JS runs.”

You will see this all over the Internet, but it is an OLD practice.

OR



Option 2 (Modern Standard): `<head>` with defer

Put it in the `<head>` *with* the defer attribute

Cleaner separation of structure and behavior

Better for 2026 since it's more aligned with the industry standard.

```
<head>
```

```
  <script src="script.js" defer></script>
```

```
</head>
```

What defer does:

- Downloads JS while HTML is parsing
- Waits to run until HTML is fully loaded
- Keeps page rendering fast
- Maintains script order

In 2026, **this is extremely common and considered clean and professional.**

What NOT to Do

```
<head>
```

```
  <script src="script.js"></script>
```

```
</head>
```

Without defer, the browser:

- Stops building the page
- Downloads JS
- Executes JS
- Then continues HTML

This slows rendering.

● Lesson 1 — “JavaScript Runs After the Page”

Older Practice but it works, just not modern

- JS can change HTML
- Code runs top to bottom
- Script goes before </body>

Type this into a code editor:

HTML Structure:

```
<body>
  <h1 id="title">Hello</h1>

  <script src="hello.js"></script>
</body>
```

Hello.js:

```
const title = document.getElementById("title");
title.textContent = "I changed the text with JS";
```

Concept Emphasis:

“The browser builds the page first...
Then JavaScript runs.”

Keep it simple.

● Lesson 2 — “What Happens If We Move It?”

Now intentionally move the script into <head> of the code editor without defer.

```
<head>
  <script src="script.js"></script>
</head>
<body>
  <h1 id="title">Hello</h1>
</body>
```

Same JS:

```
const title = document.getElementById("title");
title.textContent = "Hello World!";
```


🌟 What Happens?

ERROR:

Cannot set properties of null

Why It Breaks (Simple Explanation)

When JS runs in <head>:

The browser reads the page like this:

1. Read <head>
2. See <script>
3. STOP building HTML
4. Run JavaScript

But the <h1> does not exist yet.

So, this line:

```
document.getElementById("title")
```

returns:

null

Because the element hasn't been created yet.

❌ Script in Head (No defer)

Browser reads:

<head>

<script> → runs JS NOW ❌

<body> hasn't loaded yet ❌

<h1> doesn't exist ❌

🟦 Lesson 3 — The Modern Fix: defer

Now add the word defer to the script in the <head> section:

```
<head>
```

```
  <script src="script.js" defer></script>
```

```
</head>
```

```
<body>
```

```
  <h1 id="title">Hello</h1>
```

```
</body>
```

What defer Does:

1. Browser reads <head>
2. Sees script
3. Downloads it
4. Keeps building page
5. AFTER page is done → runs JS

Now everything works.

Without defer

HTML → STOP → JS → HTML continues

With defer

HTML → HTML → HTML → JS runs at end

“JavaScript can only change something that already exists.

If the browser hasn’t built it yet, JavaScript can’t find it.

defer tells the browser:

‘Wait until the page is built before running me.’”

Question that you might be asking yourself:

When the code is moved into the head and the error fires, why does the page still load in the browser?

The page still loads because:

JavaScript errors do NOT stop the browser from building HTML.

HTML rendering and JavaScript execution are related — but they are not the same system.

Think of It Like This

Imagine the browser as two workers:

 **Worker 1: The Builder** → builds the HTML

 **Worker 2: The Electrician** → runs JavaScript

If the electrician makes a mistake...

The house is still built.

What Actually Happens (Step by Step)

Let’s say your script is in <head> without defer.

The browser reads:

```
<head>
  <script src="script.js"></script>
</head>
<body>
  <h1 id="title">Hello</h1>
</body>
```

Timeline of Events

1 Browser starts reading HTML

It enters `<head>`

2 Browser sees `<script>`

It pauses HTML building.

3 JavaScript runs

This line executes:

```
const title = document.getElementById("title");
```

But `<h1>` hasn't been built yet. It's in the HTML below this code.

So:

```
title === null
```

Then this line runs:

```
title.textContent = "Hello World!";
```

✶ Error occurs.

❓ Important Part

The browser (HTML part) says:

“That script had an error.
Okay. Moving on.”

Then it continues building:

```
<body>
  <h1 id="title">Hello</h1>
</body>
```

So, the page renders the text.

Why Doesn't the Whole Page Crash?

Because:

- JavaScript runs in its own execution environment
- An error stops *that script*

- It does NOT stop:
 - HTML parsing
 - CSS rendering
 - The rest of the page

The browser is designed to be fault tolerant.

If one script could kill the entire page, the internet would be unusable 🤖.

Visual Diagram

Without defer

Read <head>

Run JS → ❌ error happens

Continue building HTML

Display page

Try this:

Add a console.log after the error:

```
const title = document.getElementById("title");  
title.textContent = "Hello World!";
```

console.log("Did this run?");

👉 The console.log never runs.

Why? Because **JavaScript stops executing that script at the error**. But the page is still rendered.

Even Deeper Explanation

There are two separate systems happening:

1. HTML parser
2. JavaScript engine

When JS throws an uncaught runtime error:

- The JS engine halts execution of that script
- Control returns to the browser
- The HTML parser continues

That separation is an intentional browser design.

The Big Idea :

“An error stops the JavaScript.
It does NOT stop the browser.”